JAVA 2

Module ITF

Interfaces

Au programme

1. Ce qu'une interface déclare
2. Plusieurs interfaces à la fois
3. Interface ou classe abstraite ?
4. Dépendre d'un contrat
5. Le patron Observateur
6. Ce qu'une interface contient aussi

1. Ce qu'une interface déclare



Démonstration



Démonstration

Trois mots, et le programme accepte l'inconnu

Déroulé : Demo_ITF

Déclarer une interface

Une interface énonce ce qu'une classe doit savoir faire, sans dire comment

Le mot-clé `interface` remplace le mot-clé `class`

Les méthodes s'y déclarent par leur entête, sans corps

Elles sont publiques sans qu'on l'écrive : `public` y est superflu

```
public interface Mesurable {

    double mesure();
}
```

Une classe s'engage

Le mot-clé `implements` déclare qu'une classe tient la promesse

Le compilateur vérifie que toutes les méthodes annoncées sont écrites

Deux classes sans aucun rapport peuvent tenir la même promesse

```java
public class Examen implements Mesurable {

    private final int points;

    @Override
    public double mesure() {
        return points;
    }
}

public class Colis implements Mesurable {

    private final double grammes;

    @Override
    public double mesure() {
        return grammes;
    }
}
```

Le type peut être l'interface

On déclare des variables du type de l'interface, mais on ne l'instancie pas

À travers ce type, seules les méthodes de l'interface sont visibles

C'est le polymorphisme du module ABS, sans classe mère

```
Mesurable m = new Colis(1250);
System.out.println(m.mesure());          // OK

Mesurable autre = new Mesurable();       // erreur de compilation
System.out.println(m.destinataire());    // erreur de compilation
```

Recevoir un comportement en paramètre

Une méthode qui reçoit un `Mesurable` accepte tout ce qui en est un

Y compris des classes écrites longtemps après elle

Le type du paramètre déclare le besoin, pas le fournisseur

```
public static double total(Mesurable[] choses) {
    double somme = 0;
    for (Mesurable chose : choses) {
        somme += chose.mesure();
    }
    return somme;
}
```

Exercices



ITF.1 Une interface et deux implémentations

fascicule ITF

ITF.2 Payer en ligne

fascicule ITF

Écrire : CC-MET.7

2. Plusieurs interfaces à la fois



Une seule mère, autant d'interfaces qu'on veut

Une classe Java n'hérite que d'une seule autre : pas de double `extends`

Rien n'empêche d'implémenter plusieurs interfaces

L'ordre des mots est imposé : `extends` d'abord, `implements` ensuite

```
public class Square extends Shape
        implements Translatable, Rotatable {

    // ...
}
```

Un comportement qui traverse les familles

Une classe abstraite dit ce qu'un objet est, une interface ce qu'il sait faire

Une image et un texte peuvent se déplacer sans être des formes

Aucune classe mère ne réunirait les trois sans mentir

```java
public interface Translatable {
    void translate(double dx, double dy);
}

public interface Rotatable {
    void rotate(double alpha);
}
```

Où écrire la méthode

Une classe abstraite peut implémenter l'interface pour toutes ses filles

La règle est celle du module ABS : ce qui est commun monte, ce qui varie descend

La question se pose à chaque méthode, et ITF.3 vous la pose pour de bon

```
public abstract class Shape
        implements Translatable, Rotatable {

    // abstraites, ou ecrites ici ?
}
```

Exercices

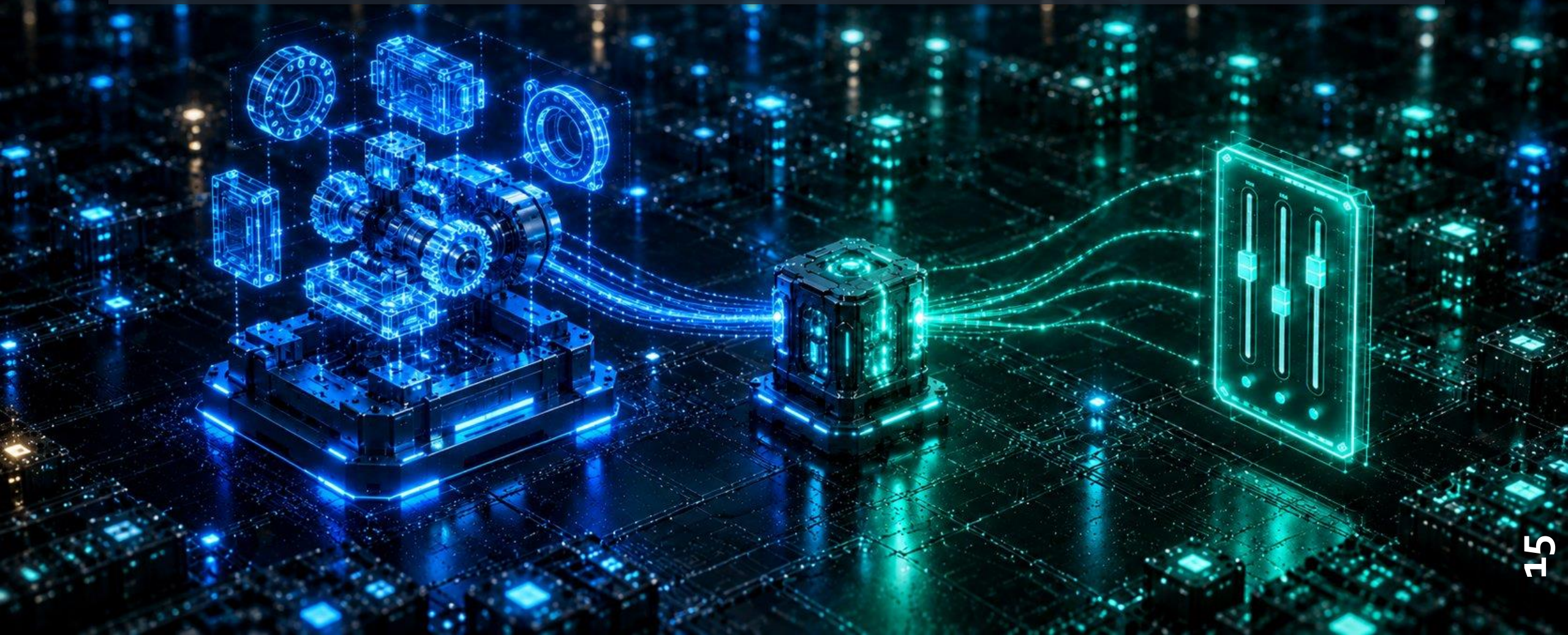


ITF.3 Des formes qui se déplacent et tournent

fascicule ITF

Les formes d'ABS.1 sont reprises ici, complètes
Concevoir : CC-CLS.3

3. Interface ou classe abstraite ?



Deux outils, deux intentions

	Classe abstraite	Interface
Contient du code ?	oui, et des champs	des signatures seulement
Combien par classe ?	une seule	autant qu'on veut
Ce qu'elle exprime	« est un »	« sait faire »
Ce qu'elle apporte	le partage du code	le découplage

Les deux ensemble

Les deux outils ne s'excluent pas

L'interface énonce ce que tout le monde sait faire

La hiérarchie partage le code, étage par étage

```
Facturable          interface : double montant()
  |
  +- Prestation     abstraite : client, facture()
      |
      +- Forfait    abstraite : prixForfait
      |   +- Abonnement
      |   +- Installation
      |
      +- Horaire    abstraite : tauxHoraire, heures
          +- Depannage
          +- Urgence
```

Exercices



ABS.4 Une hiérarchie à deux niveaux

fascicule ABS

Annoncé au module ABS, il devient faisable maintenant

4. Dépendre d'un contrat

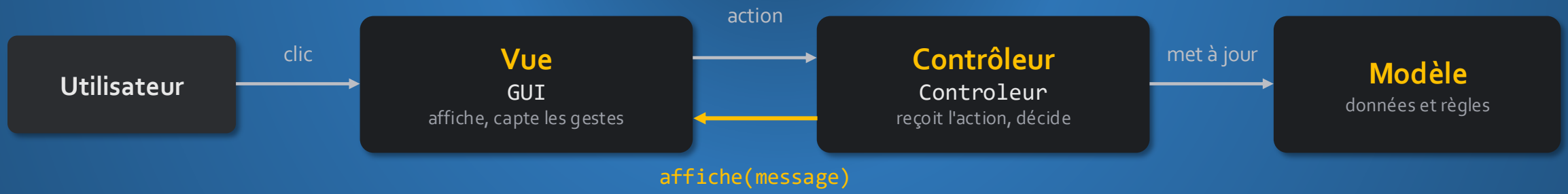


Le modèle MVC : qui fait quoi

Une application graphique se découpe souvent en trois rôles : la vue, le contrôleur et le modèle

La vue (`GUI`) affiche et capte les gestes de l'utilisateur ; le modèle contient les données et les règles

Le contrôleur fait le lien : il reçoit l'action, met le modèle à jour, et demande à la vue d'afficher



Deux classes qui se tiennent

Dans ce premier exemple, le contrôleur nomme la classe `GUI` : il ne marche qu'avec cette classe-là…

Changer d'affichage oblige à modifier le contrôleur et sa méthode `setAffiche ur`

Les deux classes ne peuvent plus être testées séparément

```
public class Controleur {

    private GUI afficheur;

    public void setAfficheur(GUI afficheur) {
        this.afficheur = afficheur;
    }
}
```

Une interface entre les deux

L'interface énonce le (seul) service dont on a besoin

Le contrôleur ne nomme plus aucune classe concrète

Affichage console, fenêtre ou trace : au choix, sans rouvrir le contrôleur

```
public interface Afficheur {
    void affiche(String message);
}

public class Controleur {

    private Afficheur afficheur;

    public void setAfficheur(Afficheur afficheur) { ... }
}
```

Le type le plus faible qui suffit

Le principe de ségrégation : ne demander que ce dont on a vraiment besoin

Choisir le type d'un paramètre, c'est choisir ce qu'on s'autorise à appeler

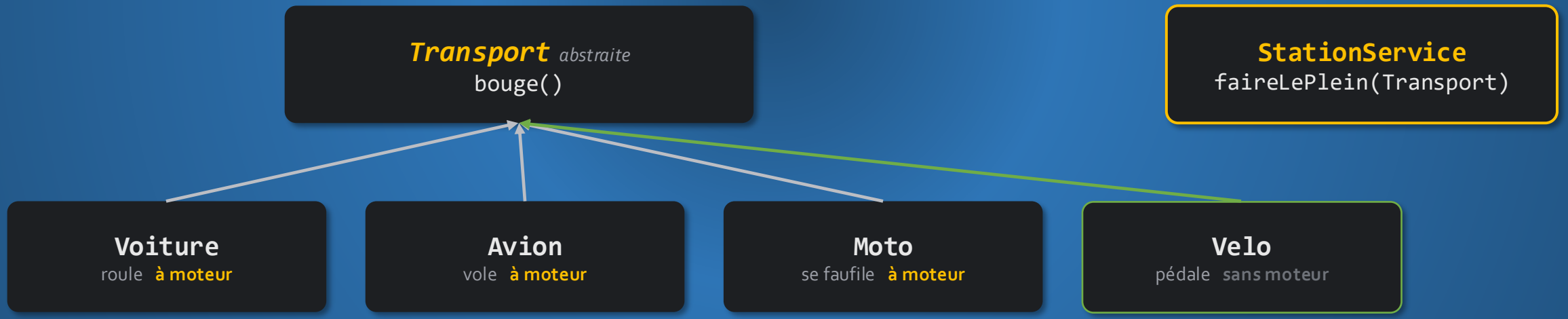
Une interface qui déclare trop de méthodes oblige ses implémentations à trop en faire

Des transports et une station-service

Une petite hiérarchie de transports : chacun sait bouger, à sa façon

Une station-service doit faire le plein des transports qu'on lui présente

Quel est l'impact dans cette application de l'ajout d'une classe Velo ?



Un comportement qui ne suit pas la hiérarchie

Faire le plein concerne la voiture, mais pas le vélo

Aucune classe mère ne dessine ce découpage

La station teste le type, et sera rouverte sans cesse

```
public class StationService {

    // a ne pas ecrire
    public void faireLePlein(Transport transport) {
        if (transport instanceof Velo) {
            // ne rien faire
        } else {
            // faire le plein
        }
    }
}
```

L'interface supprime le test

Si au lieu d'utiliser une classe mère on modélise le comportement dans une interface, seuls les transports à moteur implémentent `Motorise`

La station reçoit un `Motorise` : le vélo est refusé

Le test de type a disparu, il n'a pas été déplacé

```
public interface Motorise {
    void faisLePlein();
}

public class StationService {

    public void faireLePlein(Motorise motorise) {
        motorise.faisLePlein();
    }
}
```

5. Le patron Observateur



Un patron de conception

Un patron de conception est une solution éprouvée à un problème de conception qui revient souvent

Ce n'est ni une bibliothèque ni du code à copier : c'est une structure

On les range en trois familles : création, structure, comportement

Le problème réglé par le patron Observateur

Plusieurs objets veulent être prévenus quand quelque chose arrive en magasin

Les faire demander sans cesse l'info au magasin : coûteux, et toujours en retard d'un instant

Prévenir chacun individuellement : le magasin doit connaître chacun d'eux

Le dispositif

L'interface Abonne déclare une seule méthode

Le magasin tient un tableau d'abonnés, rien de plus

S'abonner, se désabonner, prévenir : trois méthodes

```
public class Magasin {

    private final Abonne[] abonnes = new Abonne[10];

    public void receptionne(String produit) {
        for (Abonne abonne : abonnes) {
            abonne.nouveauProduit(produit);
        }
    }
}
```

Ce que ça change

Ajouter un acteur : une classe, et rien d'autre à modifier

Le magasin ne saura jamais à qui il parle

On ne demande pas son état à l'abonné : on lui demande d'agir (`CC-OBJ.1`)

Exercices

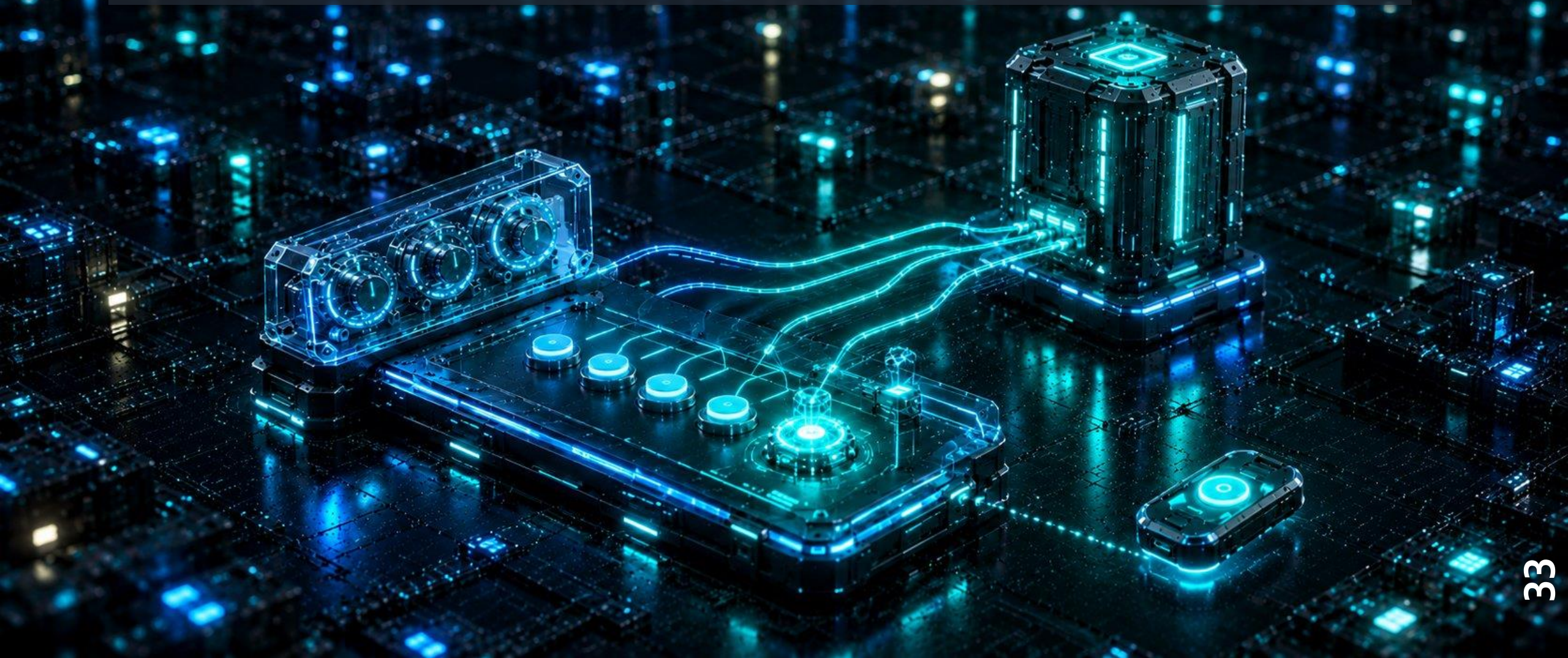


ITF.4 Prévenir sans connaître

fascicule ITF

Vous ne modifiez ni le `Magasin`, ni l'interface, ni les deux autres abonnés

6. Autres caractéristiques de l’interface



Des constantes, et des méthodes utilitaires

Un champ d'interface est implicitement `public static final`

Une interface peut contenir des méthodes `static`, corps compris

Elles s'appellent sur l'interface : `Mesurable.negligeable(colis)`

```
public interface Mesurable {

    double SEUIL = 0.001;   // public static final

    double mesure();

    static boolean negligeable(Mesurable m) {
        return m.mesure() < SEUIL;
    }
}
```

Ajouter une méthode sans casser l'existant

Une interface implémentée par dix classes ne peut pas gagner une méthode : les dix cessent de compiler

Une méthode `default` contient un corps, dont toutes les classes héritent

Chacune peut la redéfinir, aucune n'y est obligée

```
public interface Vehicle {

    void demarre();
    void arrete();

    default void klaxonne() {
        System.out.println("Tuut");
    }
}
```

Une interface sans aucune méthode

Une interface de marquage ne déclare rien : elle pose une étiquette

Elle ne dit pas « sait faire », mais « appartient à cette catégorie »

`Serializable` autorise un objet à être écrit dans un fichier, et c'est tout

```
public class Contact implements Serializable {

    // rien a implementer : l'interface ne demande rien
}
```

Une interface, une seule méthode

Une interface avec exactement une méthode abstraite est dite fonctionnelle

Vous en avez écrit deux : `Calculateur` et `Translatable`

L'annotation `@FunctionalInterface` fait vérifier la règle

Au module `LAM`, elles gagneront une écriture abrégée

```
@FunctionalInterface
public interface Calculateur {

    int calcule(int a, int b);
}
```

Le fil rouge



G.11 L'interface `ObservateurPartie`
fil rouge GameHub

G.12 La partie enregistre ses observateurs
fil rouge GameHub

G.13 Un journal qui suit la partie
fil rouge GameHub

La liste d'observateurs de `G.12` vous est donnée : les collections sont le sujet du module COL